

# Quantum-Correction

## Surface Codes, Syndrome Extraction, MWPM, and Decoder-Weight Adaptation

### Technical Report

Rotated surface-code memory with circuit-level noise, Stim sampling, and PyMatching minimum-weight perfect matching (MWPM).

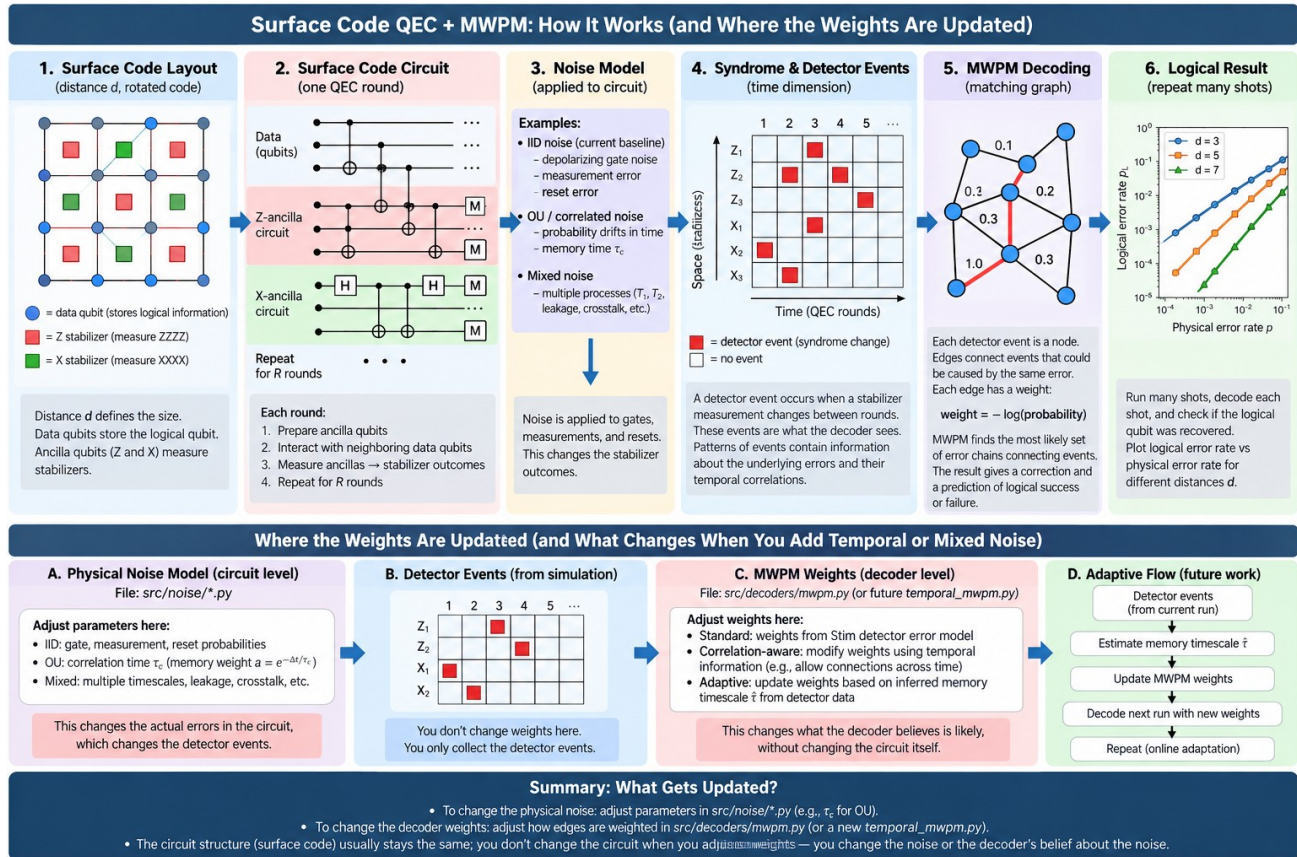


Figure 1. System-level view of the surface-code simulation, detector-event generation, MWPM decoding, and the locations at which physical-noise parameters or decoder weights are modified.

**Abstract.** This report describes the current Quantum-Correction implementation and its proposed extension to temporally correlated noise. The baseline consists of a rotated surface-code memory, independent and identically distributed (IID) circuit-level noise, Stim-based detector sampling, and PyMatching MWPM decoding. Stabilizer measurements convert hidden physical faults into classical detector events in space-time. The decoder assigns likelihood-based weights to candidate error paths and selects a minimum-weight explanation of the observed events. The principal research direction is to introduce Ornstein-Uhlenbeck (OU) noise, estimate memory timescales from detector data, and evaluate correlation-aware decoder weights without changing the surface-code circuit structure.

# Contents

1. System Overview
2. Surface-Code Representation
3. Stabilizer Measurement and Syndrome Extraction
4. Relation to Classical Parity-Check Codes
5. Circuit-Level Noise Model
6. Detector Events in Space-Time
7. MWPM Decoding and Edge Weights
8. Logical-Error-Rate Estimation
9. Temporally Correlated Noise
10. Repository Architecture and Extension Plan
11. Conclusions and Research Questions

# 1. System Overview

The implemented workflow is a sequence of physical simulation, syndrome extraction, detector construction, graph decoding, and logical-observable evaluation. Physical faults occur at specified locations in the quantum circuit. Because the data qubits cannot be measured directly without disturbing the encoded state, stabilizer measurements are repeated over multiple quantum-error-correction (QEC) rounds. Changes in these measurements form the detector data supplied to MWPM.

Processing chain: surface-code circuit → physical noise → repeated stabilizer measurements → detector events → weighted matching graph → MWPM → logical success or failure.

Stim constructs and samples the noisy circuit. PyMatching consumes the detector error model and executes MWPM. The output of one complete memory experiment is a decoded prediction of whether the encoded logical observable was preserved.

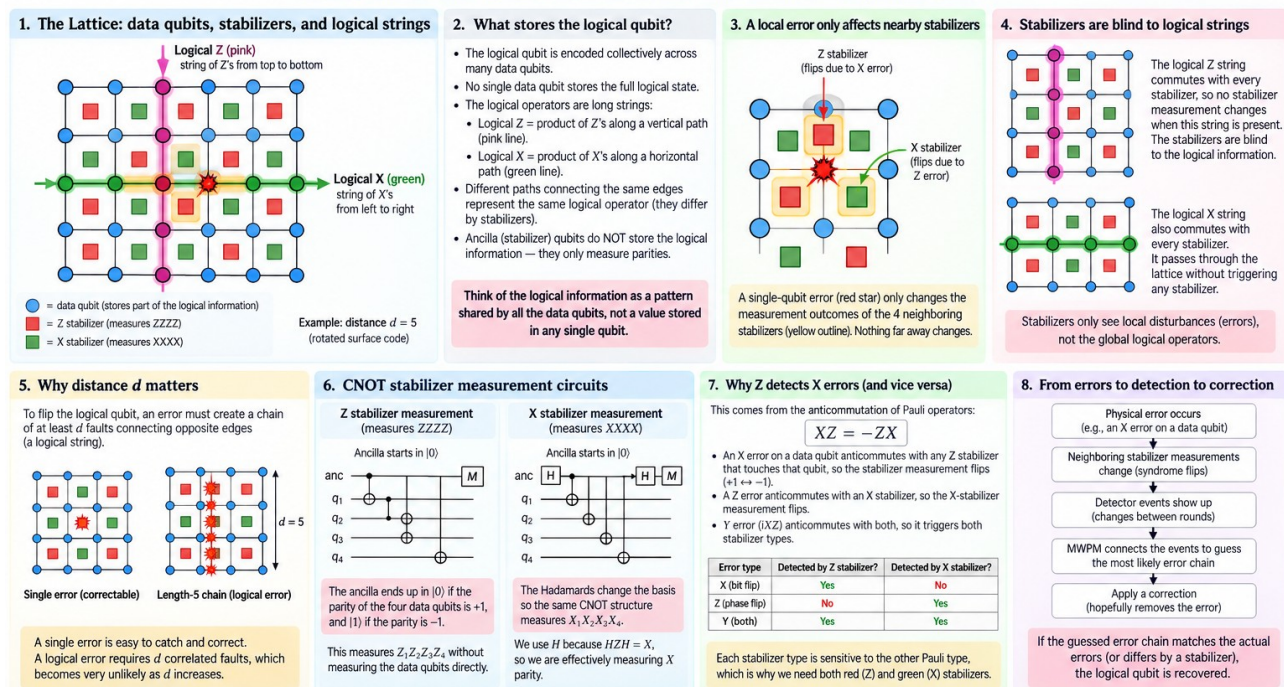
The current baseline uses a rotated surface code with IID circuit-level noise. The planned extension introduces time-correlated noise and modifies the decoder model to reflect correlations that are visible in the detector stream.

## 2. Surface-Code Representation

A surface code encodes one logical qubit nonlocally in a two-dimensional array of data qubits. No individual data qubit contains a complete copy of the logical state. Instead, the encoded information is represented by correlations across the data-qubit state. Ancilla qubits are measurement resources; they do not store the logical information.

### Understanding the Surface Code Lattice

Which qubits store the logical information, how stabilizers work, and why errors are local.



### Key Takeaway

The logical information is stored in a global pattern across many data qubits. Stabilizers (measured with CNOT circuits) only see local errors, not the logical strings. Z stabilizers detect X errors and X stabilizers detect Z errors because of anticommutation. A logical error requires a chain of length  $d$ , which becomes very unlikely as  $d$  grows.

Figure 2. Data qubits, X- and Z-type stabilizers, logical strings, local error signatures, and representative stabilizer-measurement circuits.

For a distance-3 rotated patch, nine data qubits jointly encode one logical qubit. Logical operators are represented by connected strings of Pauli operators spanning the patch. A logical-Z string connects the relevant pair of boundaries, while a logical-X string connects the complementary pair. Multiplication by stabilizers can deform a string without changing its logical action.

A stabilizer is a multi-qubit Pauli operator whose eigenvalue is measured repeatedly. Typical checks have the form  $S_X = X_1 X_2 X_3 X_4$  and  $S_Z = Z_1 Z_2 Z_3 Z_4$ . An X error anticommutes with a neighboring Z stabilizer and changes its eigenvalue; a Z error similarly changes an X stabilizer. A Y error can affect both types.

The code distance  $d$  is the minimum weight of a nontrivial logical operator. A local error chain has endpoints that produce local stabilizer changes. A chain connecting appropriate boundaries can implement a logical operation and therefore constitutes a logical failure. Increasing  $d$  increases the minimum number of coordinated faults required for such a failure.

### 3. Stabilizer Measurement and Syndrome Extraction

A QEC round prepares or resets ancilla qubits, couples each ancilla to the neighboring data qubits, measures the ancilla, and records one stabilizer outcome per check. The procedure is repeated for  $R$  rounds. The ancilla measurement extracts the parity of the selected data-qubit operators without directly measuring the logical observable.

For a Z-type stabilizer, the CNOT orientation is chosen so that the ancilla measurement records Z parity. For an X-type stabilizer, Hadamard gates place the ancilla in the X basis before and after the entangling operations. The identity  $HZH = X$  explains why the same interaction structure can measure X-type parity in a changed basis.

If an X error occurs on a data qubit contained in a Z stabilizer, the relation  $ZX = -XZ$  changes the stabilizer eigenvalue. The next ancilla measurement therefore differs from the previous result. This is an indirect error signature: the measurement reveals a parity change rather than the microscopic error operation itself.

#### Error-to-check correspondence:

| Physical error | Sensitive check    | Algebraic reason                    |
|----------------|--------------------|-------------------------------------|
| X (bit flip)   | Z-type             | X anticommutes with Z               |
| Z (phase flip) | X-type             | Z anticommutes with X               |
| Y              | Both X- and Z-type | Y contains X- and Z-like components |

The complete extraction chain is: physical fault → stabilizer-eigenvalue change → ancilla measurement change → detector event after round-to-round comparison → candidate error path for MWPM.

## 4. Relation to Classical Parity-Check Codes

The surface-code syndrome process is the quantum analogue of parity-check decoding. In a classical linear code, a parity-check matrix  $H$  maps an error vector  $e$  to a syndrome  $s = He \pmod{2}$ . The syndrome constrains the possible error locations without directly identifying the complete transmitted message.

CSS quantum codes use two compatible families of checks: one detects X-type faults and the other detects Z-type faults. The commutation condition  $H_X H_Z^T = 0$  ensures that the two families can be measured consistently. The surface code extends this parity-check structure over a local two-dimensional geometry and across repeated time rounds.

Unlike a small Hamming-code lookup table, a surface-code decoder must evaluate many possible error chains in space-time. MWPM provides a graph-based approximation in which detector events are nodes and likely fault mechanisms are weighted edges.

## 5. Circuit-Level Noise Model

The baseline applies a single probability parameter  $p$  at three circuit locations: after Clifford gates, before measurement, and after reset or preparation. This is a circuit-level model because it includes faults in the operations used to extract the syndrome, rather than restricting noise to data qubits between perfect correction cycles.

| Stim parameter                  | Physical interpretation            | Baseline |
|---------------------------------|------------------------------------|----------|
| after_clifford_depolarization   | Gate-associated depolarizing fault | $p$      |
| before_measure_flip_probability | Measurement-result flip            | $p$      |
| after_reset_flip_probability    | Reset or preparation fault         | $p$      |

The use of a common  $p$  is a calibration simplification. A hardware-informed model may assign distinct probabilities by gate type, measurement channel, reset operation, qubit, or circuit location. These parameters modify the physical fault process and consequently modify the detector statistics.

Implementation location: `src/noise/iid_noise.py`. A future OU or mixed-noise module can be added without changing the surface-code circuit definition.

## 6. Detector Events in Space-Time

Let  $s_i(t)$  denote the binary outcome of stabilizer  $i$  in round  $t$ . A detector event records a change relative to a related measurement, commonly the same stabilizer in the previous round:

$$D_i(t) = s_i(t) \text{ XOR } s_i(t - 1)$$

$D_i(t) = 0$  indicates no change;  $D_i(t) = 1$  indicates a change. The detector stream therefore contains the locations and times of parity changes, not the hidden physical fault itself. A data-qubit fault often changes neighboring stabilizers and produces a pair or small cluster of detector events. A measurement fault can produce events separated primarily in time.

The resulting decoding problem is three-dimensional: two spatial directions across the code patch and one temporal direction across QEC rounds. At a boundary, an error chain may terminate without a second interior detector; MWPM represents such termination using a boundary node.

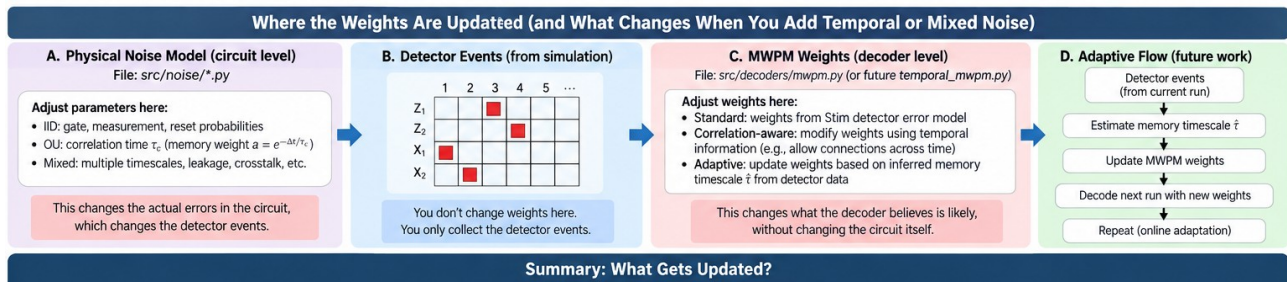


Figure 3. Separation between the physical-noise model, detector-event data, decoder weights, and a future adaptive-weighting loop.

## 7. MWPM Decoding and Edge Weights

MWPM constructs a graph whose nodes represent detector events and whose edges represent candidate mechanisms that could connect those events. The decoder selects a set of mutually compatible paths with minimum total weight. The selected paths define the most likely correction, subject to the graph model.

For an independent graph edge with probability  $p_e$ , a common log-likelihood cost is:

$$w_e = \ln((1 - p_e) / p_e)$$

For small  $p_e$ , this is closely related to  $-\ln(p_e)$ . For  $p_e = 0.01$ , the weight is approximately 4.60; for  $p_e = 0.005$ , it is approximately 5.29. Less probable mechanisms therefore receive larger costs.

The physical-noise parameters and the decoder weights are distinct model components. Changing the noise model changes which detector events are generated. Changing the decoder weights changes which candidate explanation MWPM considers most likely for a fixed detector record. A single physical fault may produce more than two detectors; Stim can decompose such mechanisms into graphlike components before PyMatching constructs the matching graph.

Implementation location: `src/decoders/mwpm.py`. The baseline obtains the detector error model from Stim; a correlation-aware implementation can modify the graph construction or edge weights while retaining the same circuit.

## 8. Logical-Error-Rate Estimation

One shot is one complete memory experiment: the encoded state is prepared, the stabilizer circuit is repeated, detector events are collected, MWPM produces a prediction, and the final data-qubit measurements determine the logical observable. A logical failure occurs when the decoded prediction disagrees with the actual logical outcome.

$$\text{logical error rate} = \text{logical failures} / \text{total shots}$$

The baseline sweeps physical error rate  $p$  and code distances  $d = 3, 5$ , and  $7$ . At sufficiently low physical error rates, increasing  $d$  should reduce the logical error rate because a longer error chain is required to implement a logical operator. The default of 10,000 shots per point is suitable for an initial baseline; rare-event estimates require substantially more shots and confidence intervals.

The logical error-rate curve is therefore a statistical estimate rather than a direct count of all physical faults. Many physical faults are successfully corrected, and only faults that produce an uncorrected logical change contribute to the logical-failure count.

## 9. Temporally Correlated Noise

IID noise assumes that the error probability in one round is independent of prior rounds. Hardware conditions may instead fluctuate gradually, producing periods in which the effective error rate remains elevated or suppressed. An OU process provides a tunable model of this memory:

$$x(t + 1) = a x(t) + \sqrt{1 - a^2} \text{ sigma } x_i(t), \quad a = \exp(-\Delta t / \tau_c)$$

Here  $\tau_c$  is the correlation time,  $a$  controls how strongly the previous state persists,  $x_i(t)$  is fresh Gaussian noise, and  $\text{sigma}$  controls fluctuation amplitude. Because  $x(t)$  is unbounded while an error probability must lie in  $[0, 1]$ , a stated mapping such as a calibrated sigmoid is required to obtain  $p(t)$ .

Temporal memory can be assessed from detector data by measuring correlations between detector activity at  $t$  and  $t + \Delta t$ . A correlation function that decays slowly indicates longer memory. The decoder extension can then incorporate time-separated connections or adaptive weights that reflect the inferred correlation structure.

## 10. Repository Architecture and Extension Plan

| Layer      | Current file                 | Current role                                            | Planned extension                         |
|------------|------------------------------|---------------------------------------------------------|-------------------------------------------|
| Circuit    | src/circuits/surface_code.py | Builds rotated surface-code memory circuit              | Usually unchanged                         |
| Noise      | src/noise/iid_noise.py       | Provides IID gate, measurement, and reset probabilities | Add OU and mixed-noise models             |
| Decoder    | src/decoders/mwpm.py         | Builds PyMatching graph from Stim DEM                   | Add temporal or correlation-aware weights |
| Experiment | experiments/baseline_mwpm.py | Sweeps distance, $p$ , and shots; saves results         | Compare IID, OU, and mixed-noise cases    |
| Tests      | tests/...                    | Validates input and baseline behavior                   | Add correlation and recovery tests        |

The current repository terminates at the IID baseline: `surface_code.py` generates the circuit, `iid_noise.py` supplies the physical noise, `Stim` generates detector data, `mwpm.py` decodes the data, and `baseline_mwpm.py` estimates logical error rates. The next implementation stage is to add OU noise, verify its detector-level memory signature, and evaluate whether decoder weights that incorporate this memory improve logical performance.

## 11. Conclusions and Research Questions

The surface-code implementation separates four concepts that must remain distinct in both analysis and software: the physical circuit, the noise process, the detector record, and the decoder model. The encoded logical information is distributed across data-qubit correlations. Stabilizers provide local parity information, and repeated measurement converts changes in those parities into detector events. MWPM then infers a likely error chain using a weighted graph.

The baseline provides a controlled reference for studying temporal correlations. Introducing OU noise changes the statistics of the physical faults and detector events; modifying the MWPM weights changes the decoder's probabilistic assumptions. The circuit structure can remain fixed while these two model layers are evaluated independently.

### Research questions:

- How faithfully do detector correlations reflect the memory of the underlying physical noise?
- Can distinct hidden noise processes with different timescales be distinguished from detector data?
- When are two physical mechanisms statistically indistinguishable at the syndrome level?
- How much does IID-weighted MWPM degrade when the true noise has temporal memory?
- Can inferred temporal structure be converted into improved edge weights or an adaptive decoder window?